

Tailwind v4 `@theme` – Full Concept + Implementation Notes

1 Purpose

- `@theme` allows defining **design tokens directly in CSS**.
- Tokens can have **light() and dark() variants** for automatic theme switching.
- Removes the need to extend colors in `tailwind.config.js`.
- Works seamlessly with **React, Next.js, or plain HTML**.

2 Syntax

```
@theme {  
  --token-name: light(#hex) dark(#hex);  
}
```

- `--token-name` → your design token (color, background, border, etc.).
- `light(#hex)` → value used when `.dark` **is absent**.
- `dark(#hex)` → value used when `.dark` **is present**.

Class mapping in Tailwind:

Token	Tailwind Class
<code>--color-text</code>	<code>text-text</code>
<code>--color-background</code>	<code>bg-background</code>
<code>--color-border</code>	<code>border-border</code>

✓ Single class works for both light and dark modes.

3 Theme Switching

3.1 Via HTML Class

- Add `.dark` on `<html>` or `<body>` → all `dark()` tokens become active.
- Remove `.dark` → light mode active.

3.2 Via JavaScript

```
const html = document.documentElement;
html.classList.toggle("dark", true); // force dark
html.classList.toggle("dark", false); // force light
```

- `.classList.toggle(className, force)` adds/removes class based on `force`.

3.3 Detect System Preference

```
if (window.matchMedia("(prefers-color-scheme: dark)").matches) {
  document.documentElement.classList.add("dark");
}
```

- Ensures theme follows user OS preference.

4 Full Implementation in Next.js / React

4.1 `globals.css` – Define Tokens

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@theme {
  /* Backgrounds */
  --color-background: light(#f2f2f2) dark(#1f1f1f);
  --color-surface: light(#ffffff) dark(#2a2828);

  /* Text */
  --color-text: light(#303030) dark(#e0e0e0);
  --color-text-muted: light(#666666) dark(#a1a1a1);
  --color-text-inverse: light(#f2f2f2) dark(#171717);

  /* Accents */
  --color-accent-blue: light(#68b5fb) dark(#508cc3);
  --color-accent-pink: light(#c54276) dark(#f75f8f);

  /* Borders */
  --color-border: light(#e6e6e6) dark(#797979);
}
```

4.2 useTheme.js – React Hook

```
import { useEffect, useState } from "react";

export default function useTheme() {
  const [theme, setTheme] = useState("light");

  useEffect(() => {
    const html = document.documentElement;
    const prefersDark = window.matchMedia("(prefers-color-scheme: dark)").matches;
    setTheme(prefersDark ? "dark" : "light");
    html.classList.toggle("dark", prefersDark);

    const mediaQuery = window.matchMedia("(prefers-color-scheme: dark)");
    const handleChange = (e) => {
      const newTheme = e.matches ? "dark" : "light";
      setTheme(newTheme);
      html.classList.toggle("dark", e.matches);
    };
    mediaQuery.addEventListener("change", handleChange);

    return () => mediaQuery.removeEventListener("change", handleChange);
  }, []);

  const toggleTheme = () => {
    const html = document.documentElement;
    const newTheme = theme === "light" ? "dark" : "light";
    html.classList.toggle("dark", newTheme === "dark");
    setTheme(newTheme);
  };

  return [theme, toggleTheme];
}
```

4.3 ThemeExample.js – React Component

```
import useTheme from "@/hooks/useTheme";

export default function ThemeExample() {
  const [theme, toggleTheme] = useTheme();

  return (
    <div className="bg-background text-text min-h-screen flex flex-col items-center justify-center">
      <h1 className="text-accent-blue text-3xl mb-4">Hello Novix 
```

```

    <p className="text-text-muted mb-6">Current theme: {theme.toUpperCase()}</
p>
    <button
      onClick={toggleTheme}
      className="px-4 py-2 bg-accent-pink text-text-inverse rounded-lg
hover:opacity-80 transition"
    >
      Toggle Theme
    </button>
    <div className="mt-8 p-4 border border-border rounded-lg bg-surface">
      <p className="text-text-inverse">This box also switches automatically!</
p>
    </div>
  </div>
);
}

```

4.4 Usage in `pages/index.js`

```

import ThemeExample from "@components/ThemeExample";

export default function Home() {
  return <ThemeExample />;
}

```

5 Key Points

- **Single token per color** → automatically switches light/dark.
- **No `` prefixes** needed in Tailwind classes.
- `.dark` class on `<html>` triggers dark mode.
- JS toggle and system preference detection make theme **dynamic**.
- Fully compatible with React / Next.js.

JAI SHREE RAM!